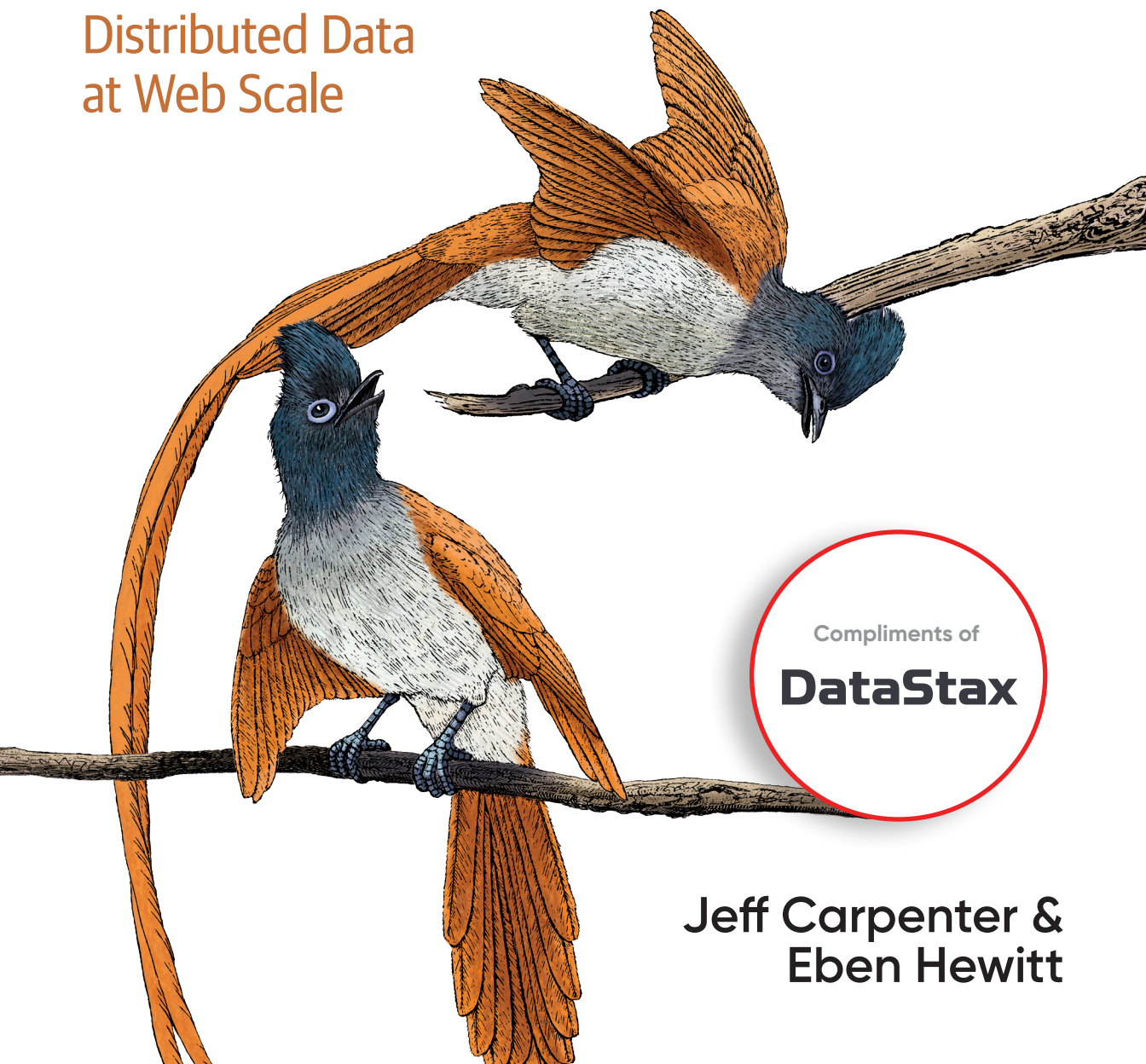


O'REILLY®

Cassandra

The Definitive Guide

Distributed Data
at Web Scale



Compliments of
DataStax

Jeff Carpenter &
Eben Hewitt

Cassandra: The Definitive Guide

Imagine what you could do if scalability wasn't a problem. With this hands-on guide, you'll learn how the Cassandra database management system handles hundreds of terabytes of data while remaining highly available across multiple data centers. This revised third edition—updated for Cassandra 4.0 and new developments in the Cassandra ecosystem, including deployments in Kubernetes with K8ssandra—provides technical details and practical examples to help you put this database to work in a production environment.

Authors Jeff Carpenter and Eben Hewitt demonstrate the advantages of Cassandra's nonrelational design, with special attention to data modeling. Developers, DBAs, and application architects looking to solve a database scaling issue or future-proof an application will learn how to harness Cassandra's speed and flexibility.

- Understand Cassandra's distributed and decentralized structure
- Use the Cassandra Query Language (CQL) and cqlsh—the CQL shell
- Create a working data model and compare it with an equivalent relational model
- Design and develop applications using client drivers
- Explore cluster topology and learn how nodes exchange data
- Maintain a high level of performance in your cluster
- Deploy Cassandra onsite, in the cloud, or with Docker and Kubernetes
- Integrate Cassandra with Spark, Kafka, Elasticsearch, Solr, and Lucene

"This book will teach you why and how to apply Cassandra in your application. Build something amazing and be yet another success story."

—Patrick McFadin
VP Developer Relations at DataStax

Jeff Carpenter is a software engineer and developer advocate at DataStax, where he leverages his background in system architecture, microservices, and Apache Cassandra to help empower developers and operations engineers to build distributed systems that are scalable, reliable, and secure.

Eben Hewitt is the CTO and chief architect at Sabre Hospitality, where he's responsible for the technology strategy and for designing large-scale, mission-critical systems and leading teams to build them. He has served as CTO at one of the world's largest hotel companies and as CIO of O'Reilly.

DATA

Twitter: @oreillymedia
linkedin.com/company/oreilly-media
youtube.com/oreillymedia



REVISED THIRD EDITION

Cassandra: The Definitive Guide

Distributed Data at Web Scale

Jeff Carpenter and Eben Hewitt

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Cassandra: The Definitive Guide

by Jeff Carpenter and Eben Hewitt

Copyright © 2022 Jeff Carpenter. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Jess Haberman**Development Editor:** Sarah Grey**Production Editor:** Deborah Baker**Copyeditor:** Sonia Saruba**Proofreader:** Kim Cofer**Indexer:** Potomac Indexing, LLC**Interior Designer:** David Futato**Cover Designer:** Karen Montgomery**Illustrator:** Rebecca Demarest

December 2010: First Edition

July 2016: Second Edition

April 2020: Third Edition

January 2022: Revised Third Edition

Revision History for the Revised Third Edition

2022-01-21: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492097143> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Cassandra: The Definitive Guide*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and DataStax. See our [statement of editorial independence](#).

978-1-492-09775-4

[LSI]

*This book is dedicated to my sweetheart, Alison Brown. I can hear the sound of violins,
long before it begins.*

—E.H.

For Stephanie, my inspiration, unfailing support, and the love of my life.

—J.C.

Table of Contents

Foreword.....	xv
Preface.....	xvii
1. Beyond Relational Databases.....	1
What's Wrong with Relational Databases?	1
A Quick Review of Relational Databases	5
Transactions, ACID-ity, and Two-Phase Commit	6
Schema	9
Sharding and Shared-Nothing Architecture	10
Web Scale	12
The Rise of NoSQL	13
Summary	16
2. Introducing Cassandra.....	17
The Cassandra Elevator Pitch	17
Cassandra in 50 Words or Less	17
Distributed and Decentralized	18
Elastic Scalability	19
High Availability and Fault Tolerance	19
Tuneable Consistency	20
Brewer's CAP Theorem	23
Row-Oriented	27
High Performance	28
Where Did Cassandra Come From?	28
Is Cassandra a Good Fit for My Project?	30
Large Deployments	30
Lots of Writes, Statistics, and Analysis	30

Geographical Distribution	31
Hybrid Cloud and Multicloud Deployment	31
Getting Involved	31
Summary	34
3. Installing Cassandra.....	35
Installing the Apache Distribution	35
Extracting the Download	36
What's in There?	36
Building from Source	38
Additional Build Targets	39
Running Cassandra	39
Setting the Environment	40
Starting the Server	41
Stopping Cassandra	43
Other Cassandra Distributions	44
Running the CQL Shell	45
Basic cqlsh Commands	46
cqlsh Help	46
Describing the Environment in cqlsh	47
Creating a Keyspace and Table in cqlsh	48
Writing and Reading Data in cqlsh	51
Running Cassandra in Docker	53
Summary	54
4. The Cassandra Query Language.....	55
The Relational Data Model	55
Cassandra's Data Model	56
Clusters	59
Keyspaces	59
Tables	59
Columns	63
CQL Types	66
Numeric Data Types	67
Textual Data Types	67
Time and Identity Data Types	68
Other Simple Data Types	70
Collections	72
Tuples	75
User-Defined Types	76
Summary	79

5. Data Modeling.....	81
Conceptual Data Modeling	81
RDBMS Design	82
Design Differences Between RDBMS and Cassandra	83
Defining Application Queries	86
Logical Data Modeling	87
Hotel Logical Data Model	89
Reservation Logical Data Model	91
Physical Data Modeling	93
Hotel Physical Data Model	94
Reservation Physical Data Model	96
Evaluating and Refining	97
Calculating Partition Size	97
Calculating Size on Disk	98
Breaking Up Large Partitions	100
Defining Database Schema	101
Cassandra Data Modeling Tools	103
Summary	106
 6. The Cassandra Architecture.....	 107
Data Centers and Racks	107
Gossip and Failure Detection	108
Snitches	110
Rings and Tokens	111
Virtual Nodes	113
Partitioners	113
Replication Strategies	114
Consistency Levels	115
Queries and Coordinator Nodes	117
Hinted Handoff	118
Anti-Entropy, Repair, and Merkle Trees	119
Lightweight Transactions and Paxos	120
Memtables, SSTables, and Commit Logs	122
Bloom Filters	124
Caching	125
Compaction	125
Deletion and Tombstones	127
Managers and Services	128
Cassandra Daemon	128
Storage Engine	129
Storage Service	129
Storage Proxy	130

Messaging Service	130
Stream Manager	130
CQL Native Transport Server	131
System Keyspaces	131
Summary	134
7. Designing Applications with Cassandra.....	135
Hotel Application Design	135
Cassandra and Microservice Architecture	135
Microservice Architecture for a Hotel Application	137
Identifying Bounded Contexts	138
Identifying Services	138
Designing Microservice Persistence	140
Extending Designs	143
Secondary Indexes	144
Materialized Views	148
Reservation Service: A Sample Microservice	152
Design Choices for a Java Microservice	152
Deployment and Integration Considerations	153
Services, Keyspaces, and Clusters	153
Data Centers and Load Balancing	154
Interactions Between Microservices	154
Summary	156
8. Application Development with Drivers.....	157
DataStax Java Driver	158
Development Environment Configuration	158
Connecting to a Cluster	159
Statements	161
Simple Statements	162
Prepared Statements	163
Query Builder	165
Object Mapper	167
Asynchronous Execution	170
Driver Configuration	172
Metadata	178
Debugging and Monitoring	180
DataStax Python Driver	181
DataStax Node.js Driver	182
DataStax C# Driver	183
Other Cassandra Drivers	183
Summary	185

9. Writing and Reading Data.....	187
Writing	187
Write Consistency Levels	187
The Cassandra Write Path	189
Writing Files to Disk	191
Lightweight Transactions	193
Batches	196
Reading	199
Read Consistency Levels	199
The Cassandra Read Path	201
Read Repair	203
Range Queries, Ordering and Filtering	205
Paging	208
Deleting	210
Summary	212
10. Configuring and Deploying Cassandra.....	213
Cassandra Cluster Manager	213
Creating a Cluster	214
Adding Nodes to a Cluster	217
Dynamic Ring Participation	218
Node Configuration	219
Seed Nodes	219
Snitches	220
Partitioners	222
Tokens and Virtual Nodes	224
Network Interfaces	225
Data Storage	226
Startup and JVM Settings	227
Planning a Cluster Deployment	229
Cluster Topology and Replication Strategies	229
Sizing Your Cluster	232
Selecting Instances	234
Storage	234
Network	235
Cloud Deployment	236
Amazon Web Services	236
Google Cloud Platform	239
Microsoft Azure	240
Summary	241

11. Monitoring.....	243
Monitoring Cassandra with JMX	243
Cassandra's MBeans	246
Database MBeans	247
Cluster-Related MBeans	250
Internal MBeans	252
Monitoring with nodetool	253
Getting Cluster Information	254
Getting Statistics	257
Virtual Tables	259
System Virtual Schema	260
System Views	261
Metrics	263
Logging	266
Examining Log Files	268
Full Query Logging	270
Summary	271
 12. Maintenance.....	 273
Health Check	273
Common Maintenance Tasks	274
Flush	274
Cleanup	275
Repair	276
Rebuilding Indexes	281
Moving Tokens	281
Adding Nodes	282
Adding Nodes to an Existing Data Center	282
Adding a Data Center to a Cluster	283
Handling Node Failure	284
Repairing Failed Nodes	285
Replacing Nodes	286
Removing Nodes	287
Upgrading Cassandra	290
Backup and Recovery	292
Taking a Snapshot	293
Clearing a Snapshot	294
Enabling Incremental Backup	294
Restoring from Snapshot	295
SSTable Utilities	296
Maintenance Tools	297
Netflix Priam	297

DataStax OpsCenter	298
Cassandra Sidecars	298
Cassandra Kubernetes Operators	299
Summary	303
13. Performance Tuning	305
Managing Performance	305
Setting Performance Goals	305
Benchmarking and Stress Testing	307
Monitoring Performance	312
Analyzing Performance Issues	313
Tracing	314
Tuning Methodology	316
Caching	317
Key Cache	317
Row Cache	318
Chunk Cache	319
Counter Cache	319
Saved Cache Settings	320
Memtables	320
Commit Logs	321
SSTables	323
Hinted Handoff	324
Compaction	325
Concurrency and Threading	328
Networking and Timeouts	329
JVM Settings	331
Memory	332
Garbage Collection	332
Summary	336
14. Security	337
Authentication and Authorization	339
Password Authenticator	339
Using CassandraAuthorizer	342
Role-Based Access Control	344
Encryption	346
SSL, TLS, and Certificates	347
Node-to-Node Encryption	350
Client-to-Node Encryption	352
JMX Security	353
Securing JMX Access	353

Security MBeans	355
Audit Logging	356
Summary	359
15. Migrating and Integrating.....	361
Knowing When to Migrate	361
Adapting the Data Model	362
Translating Entities	363
Translating Relationships	365
Adapting the Application	367
Refactoring Data Access	368
Maintaining Consistency	368
Migrating Stored Procedures	370
Planning the Deployment	375
Migrating Data	376
Zero-Downtime Migration	376
Bulk Loading	377
Common Integrations	379
Managing Data Flow with Apache Kafka	379
Searching with Apache Lucene, SOLR, and Elasticsearch	382
Analyzing Data with Apache Spark	382
Summary	389
Index.....	391

Foreword

I first came across Apache Cassandra at the start of 2009. As a beta tester for push alerts on the iPhone, our mobile news application was having a difficult time handling millions of user lookups to send timely updates. After evaluating several different approaches, I stumbled onto Cassandra. Here was a database where I had to model my data for the query I was going to make, optimizing the “fetch” to return the whole result set in a single disk seek. What had been taking up to 30 minutes, impacting latencies across the site, sped up to a matter of seconds—so fast that our development contacts at Apple had to ask us to lower our throughput!

Almost every aspect of Cassandra has changed substantially since then. What was initially a niche group of power users and distributed systems enthusiasts has blossomed into a thriving, diverse community. Over the years, Cassandra has proved itself time and again, running some of the largest workloads on the internet for services you use every day. An equally valid maturity signal is that you now have in front of you a third edition of this book.

Through all its successes, Cassandra is still a difficult system to use. From installation to integration and operationalization, it remains nuanced, with plenty of gotchas. This book does a fantastic job of walking the reader through common pitfalls, with detailed explanations of important concepts.

No matter what your focus, read all the way through this book, so you understand the Cassandra system as a whole. As a developer, you may never need to care about anti-entropy repair (that’s the ops team’s job!), but you still need to understand its impact on maintaining data consistency. This may be the first time you have used a system where the operational mechanics affect how you configure your application for data consistency. You’ll also need to communicate the failure boundaries and cluster topology to your application team so they can configure data distribution and consistency levels correctly.

Regardless of your role, these concepts are difficult. This book, more than any single resource I’ve come across to date, does an excellent job of explaining things. We all

learn differently, though, and I encourage you to supplement what this volume offers by engaging with Apache Cassandra’s thriving community, full of experienced power users, developers, and application architects who contribute code and documentation, participate in discussion lists and Slack channels, and speak at meetups and events. Use this book as your gateway into the world of Apache Cassandra.

— *Nate McCall*
Vice President, Apache Cassandra
Apache Software Foundation
Raumati South, New Zealand
March 25, 2020

Why Apache Cassandra?

Apache Cassandra is a free, open source, distributed data storage system that differs sharply from relational database management systems (RDBMSs).

Cassandra first started as an Incubator project at Apache in January of 2009. Shortly thereafter, the committers, led by Apache Cassandra Project Chair Jonathan Ellis, released version 0.3 of Cassandra, and steadily made releases up to the milestone 3.0 release. Since 2017, the project has been led by Apache Cassandra Project Chair Nate McCall, producing releases 3.1 through the latest 4.0 release. Cassandra is being used in production by some of the biggest companies on the web, including Facebook, Twitter, and Netflix.

Its popularity is due in large part to the outstanding technical features it provides. It is durable, seamlessly scalable, and tuneably consistent. It performs blazingly fast writes, can store hundreds of terabytes of data, and is decentralized and symmetrical so there's no single point of failure. It is highly available and offers a data model based on the Cassandra Query Language (CQL).

Is This Book for You?

This book is intended for a variety of audiences. It should be useful to you if you are:

- A developer working with large-scale, high-volume applications, such as Web 2.0 social applications, ecommerce sites, financial services, or sensor-based Internet of Things (IoT) systems
- An application architect or data architect who needs to understand the available options for high-performance, decentralized, elastic data stores

- A database administrator or database developer currently working with standard relational database systems who needs to understand how to implement a fault-tolerant, eventually consistent data store
- A manager who wants to understand the advantages (and disadvantages) of Cassandra to help make decisions about technology strategy
- A student, analyst, or researcher who is designing a project related to Cassandra or other nonrelational data store options

This book is a technical guide. In many ways, Cassandra and other NoSQL databases represent a new way of thinking about data. Many developers who gained their professional chops in the last 15–20 years have become well versed in thinking about data in purely relational or object-oriented terms. Cassandra’s data model is different and can be difficult to wrap your mind around at first, especially for those of us with entrenched ideas about what a database is (and should be).

Using Cassandra does not mean that you have to be a Java developer. However, Cassandra is written in Java, so if you’re going to dive into the source code, a solid understanding of Java is crucial. Many of the examples in this book are in Java, but Cassandra drivers are available in a wide variety of languages, including Java, Node.js, Python, C#, PHP, Ruby, and Go.

Finally, it is assumed that you have a good understanding of how the web works, can use an integrated development environment (IDE), and are somewhat familiar with the typical concerns of data-driven applications. You might be a well-seasoned developer or administrator but still, on occasion, encounter tools used in the Cassandra world that you’re not familiar with. For example, Apache Ant is used to build Cassandra, and the Cassandra source code is available via Git. In cases where we speculate that you’ll need to do a little setup of your own in order to work with the examples, we try to support that.

What’s in This Book?

This book is designed with the chapters acting, to a reasonable extent, as standalone guides. This is important for a book on Cassandra, which has a variety of audiences in different job roles and industries. To borrow from the software world, the book is designed to be modular. If you’re new to Cassandra, it makes sense to read the book in order; if you’ve passed the introductory stages, you will still find value in later chapters, which you can read as standalone guides.

Here is how the book is organized:

Chapter 1, “Beyond Relational Databases”

This chapter reviews the history of the enormously successful relational database and the rise of nonrelational database technologies like Cassandra.

Chapter 2, “Introducing Cassandra”

This chapter introduces Cassandra and discusses what’s exciting and different about it, where it came from, and what its advantages are.

Chapter 3, “Installing Cassandra”

This chapter walks you through installing Cassandra, getting it running, and trying out some of its basic features.

Chapter 4, “The Cassandra Query Language”

Here we look at Cassandra’s data model, highlighting how it differs from the traditional relational model. We also explore how this data model is expressed in the Cassandra Query Language (CQL).

Chapter 5, “Data Modeling”

This chapter introduces principles and processes for data modeling in Cassandra. We analyze a well-understood domain to produce a working schema.

Chapter 6, “The Cassandra Architecture”

This chapter helps you understand what happens during read and write operations and how the database accomplishes some of its notable aspects, such as durability and high availability. We go under the hood to understand some of the more complex inner workings, such as the gossip protocol, hinted handoffs, read repairs, Merkle trees, and more.

Chapter 7, “Designing Applications with Cassandra”

In order to help make some of Cassandra’s architecture concepts more concrete, we’ll explore some of the common ways in which Cassandra figures into the architecture and design of modern cloud applications.

Chapter 8, “Application Development with Drivers”

There are a variety of drivers available for different languages, including Java, Node.js, Python, Ruby, C#, and PHP, in order to abstract Cassandra’s lower-level API. We help you understand how to use common driver features to develop applications with Cassandra.

Chapter 9, “Writing and Reading Data”

We build on the previous chapters to learn how Cassandra works “under the covers” to read and write data. We’ll also discuss concepts such as batches, lightweight transactions, and paging.

Chapter 10, “Configuring and Deploying Cassandra”

This chapter shows you how to specify partitioners, replica placement strategies, and snitches. We set up a cluster and see the implications of different configuration choices. We’ll discuss how to plan your cluster deployments, including hybrid and multicloud deployments using providers such as Amazon, Microsoft,

and Google, as well as deploying and managing clusters using Docker and Kubernetes.

Chapter 11, “Monitoring”

Once your cluster is up and running, you’ll want to monitor its usage, memory patterns, and thread patterns, and understand its general activity. Cassandra has a rich Java Management Extensions (JMX) interface baked in, which we put to use to monitor all of these and more.

Chapter 12, “Maintenance”

The ongoing maintenance of a Cassandra cluster is made somewhat easier by some tools that ship with the server. We see how to decommission a node, load balance the cluster, get statistics, and perform other routine operational tasks.

Chapter 13, “Performance Tuning”

One of Cassandra’s most notable features is its speed—it’s very fast. But there are a number of things, including memory settings, data storage, hardware choices, caching, and buffer sizes, that you can tune to squeeze out even more performance.

Chapter 14, “Security”

NoSQL technologies are often slighted as being weak on security. Thankfully, Cassandra provides authentication, authorization, and encryption features, which we’ll learn how to configure in this chapter.

Chapter 15, “Migrating and Integrating”

We close the book with a summary of the steps involved in bringing Cassandra into your enterprise, from the perspective of migrating from a relational database to Cassandra. We’ll look at the implications for data modeling, application development, and deployment as well as how Cassandra integrates with other popular technologies, including:

- Streaming systems such as Apache Kafka
- Search engines such as Apache Lucene, Apache Solr, and Elasticsearch
- Analytics platforms such as Apache Spark



Cassandra Versions Used in This Book

This book was developed using Apache Cassandra 4.0 and the DataStax Java Driver version 4.1. The formatting and content of tool output, log files, configuration files, and error messages are as they appear in the 4.0 release, and may change in future releases.

When discussing features added in releases 2.0 and later, we cite the release in which the feature was added for readers who may be using earlier versions and are considering whether to upgrade.

New for the Third Edition

For this third edition, there is not quite as much of a time gap to cover as there was between the first and second editions, but there have been several key changes we'd like to note:

A grown-up database

The conventional wisdom in the software engineering community has been that it takes 5–10 years for a new database engine to fully mature. Thankfully, Cassandra has reached this maturity milestone, and while the 4.0 release certainly has some stability and availability improvements, the bulk of the new features are focused on features that make the database easier to understand and maintain. This edition covers new 4.0 features including: virtual tables (covered in [Chapter 11](#)), audit logging (covered in [Chapter 14](#)), and change data capture (CDC) (covered in [Chapter 15](#)).

Cassandra in cloud applications

The types of applications in which Cassandra is used continue to increase. To help bridge the gap between concept and reality, we've added a new chapter on this, [Chapter 7](#). We've also updated [Chapter 15](#) to include discussion of several patterns for using Kafka and Cassandra together.

Changes in cloud deployment

When the second edition was published, Docker had already become a popular choice for application deployment, but the verdict was still out on running databases on Docker. Since then, there have been sufficient advances that we now feel comfortable recommending deployment of Cassandra on Docker. Kubernetes has emerged as the key technology for orchestrating the deployment and maintenance of containers across clusters of machines. In this edition we've updated [Chapter 10](#) with new guidance on deployment of Cassandra to Docker and added coverage of Kubernetes to reflect the changing landscape.

Note on the Revised Third Edition

This revised third edition contains several updates to help bring the Cassandra story up to date as of 2022:

- The Cassandra community has continued to innovate since the 4.0 release. This edition highlights efforts under the Apache project umbrella, as reflected in the Cassandra Enhancement Proposal (CEP) process. We also highlight companion ventures, including [K8ssandra](#), a project for deploying Cassandra in Kubernetes.

- The experience of developing applications with Cassandra continues to improve. We've extended our coverage of development with Cassandra in **Chapter 8**, including API access with **Stargate** and driver examples in additional popular languages.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

The code examples found in this book are available for download at <https://github.com/jeffreyscarpenter/cassandra-guide> and <https://github.com/jeffreyscarpenter/reservation-service>.

This book is here to help you get your job done. In general, you may use the code in this book in your programs and documentation. You do not need to contact us for

permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Cassandra: The Definitive Guide*, Revised Edition, by Jeff Carpenter and Eben Hewitt (O'Reilly). Copyright 2022 Jeff Carpenter, 978-1-492-09714-3.”

If you feel your use of code examples falls outside fair use or the permission given here, feel free to contact us at permissions@oreilly.com.

O'Reilly Interactive Katacoda Scenarios

Interactive Katacoda scenarios mimic real-world production environments and let you write and run code as you learn, right in your browser. The author has developed a collection of Katacoda scenarios to give you hands-on practice with the tools and practices outlined in this book. Visit <http://oreilly.com> for more information about our interactive content, to view the ebook format for this title, and also to learn about all our learning platform has to offer.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, please visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North

Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at https://oreil.ly/cassandra_revisedEd.

Send an email to bookquestions@oreilly.com to comment or ask technical questions about this book.

For more information about our books, courses, and news, see our website at <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

There are many wonderful people to whom we are grateful for helping bring this book to life.

Thank you to our technical reviewers: Stu Hood, Robert Schneider, and Gary Dusba-bek contributed thoughtful reviews to the first edition, while Andrew Baker, Ewan Elliot, Kirk Damron, Corey Cole, Jeff Jirsa, Chris Judson, and Patrick McFadin reviewed the second edition. The third edition was reviewed by Pankaj Gallar, Cedrick Lunven, Alex Ott, and Wei Deng.

Thank you to Jonathan Ellis and Patrick McFadin for writing forewords for the first and second editions, respectively, and to Nate McCall for the third edition foreword. Thanks also to Patrick for his contributions to the Spark integration section in [Chapter 15](#).

Thanks to our editors, Mike Loukides, Marie Beaugureau, Nicole Taché, Jess Haberman, and Sarah Grey, for their constant support and making this a better book.

Jeff would like to thank Eben for entrusting him with the opportunity to update such a well-regarded, foundational text, and for Eben's encouragement from start to finish.

Finally, we've been inspired by the many terrific developers who have contributed to Cassandra. Hats off for making such an elegant and powerful database.

Beyond Relational Databases

If at first the idea is not absurd, then there is no hope for it.

—Albert Einstein

Welcome to *Cassandra: The Definitive Guide*. The aim of this book is to help developers and database administrators understand this important database technology. During the course of this book, we will explore how Cassandra compares to traditional relational database management systems, and help you put it to work in your own environment.

What's Wrong with Relational Databases?

If I had asked people what they wanted, they would have said faster horses.

—Henry Ford

We ask you to consider a certain model for data, invented by a small team at a company with thousands of employees. It was accessible over a TCP/IP interface and was available from a variety of languages, including Java and web services. This model was difficult at first for all but the most advanced computer scientists to understand, until broader adoption helped make the concepts clearer. Using the database built around this model required learning new terms and thinking about data storage in a different way. But as products sprang up around it, more businesses and government agencies put it to use, in no small part because it was fast—capable of processing thousands of operations a second. The revenue it generated was tremendous.

And then a new model came along.

The new model was threatening, chiefly for two reasons. First, the new model was very different from the old model, which it pointedly controverted. It was threatening because it can be hard to understand something different and new. Ensuing debates

can help entrench people stubbornly further in their views—views that might have been largely inherited from the climate in which they learned their craft and the circumstances in which they work. Second, and perhaps more importantly, as a barrier, the new model was threatening because businesses had made considerable investments in the old model and were making lots of money with it. Changing course seemed ridiculous, even impossible.

Of course, we are talking about the Information Management System (IMS) hierarchical database, invented in 1966 at IBM.

IMS was built for use in the Saturn V moon rocket. Its architect was Vern Watts, who dedicated his career to it. Many of us are familiar with IBM's database DB2. IBM's wildly popular DB2 database gets its name as the successor to DB1—the product built around the hierarchical data model IMS. IMS was released in 1968, and subsequently enjoyed success in Customer Information Control System (CICS) and other applications. It is still used today.

But in the years following the invention of IMS, the new model, the disruptive model, the threatening model, was the relational database.

In his 1970 paper, “[A Relational Model of Data for Large Shared Data Banks](#)”, Dr. Edgar F. Codd also advanced his theory of the relational model for data while working at IBM's San Jose research laboratory. This paper became the foundational work for relational database management systems.

Codd's work was antithetical to the hierarchical structure of IMS. Understanding and working with a relational database required learning new terms, including *relations*, *tuples*, and *normal form*, all of which must have sounded very strange indeed to users of IMS. It presented certain key advantages over its predecessor, such as the ability to express complex relationships between multiple entities, well beyond what could be represented by hierarchical databases.

While these ideas and their application have evolved in four decades, the relational database still is clearly one of the most successful software applications in history. It's used in the form of Microsoft Access in sole proprietorships, and in giant multinational corporations with clusters of hundreds of finely tuned instances representing multiterabyte data warehouses. Relational databases store invoices, customer records, product catalogs, accounting ledgers, user authentication schemes—the very world, it might appear. There is no question that the relational database is a key facet of the modern technology and business landscape, and one that will be with us in its various forms for many years to come, as will IMS in its various forms. The relational model presented an alternative to IMS, and each has its uses.

So the short answer to the question, “[What's Wrong with Relational Databases?](#)” is “Nothing.”

There is, however, a rather longer answer, which says that every once in a while an idea is born that ostensibly changes things, and engenders a revolution of sorts. And yet, in another way, such revolutions, viewed structurally, are simply history's business as usual. IMS, RDBMS, NoSQL. The horse, the car, the plane. They each build on prior art, they each attempt to solve certain problems, and so they're each good at certain things—and less good at others. They coexist, even now.

So let's examine for a moment why you might consider an alternative to the relational database, just as Codd himself four decades ago looked at the Information Management System and thought that maybe it wasn't the only legitimate way of organizing information and solving data problems, and that maybe, for certain problems, it might prove fruitful to consider an alternative.

You encounter scalability problems when your relational applications become successful and usage goes up. The need to gather related data from multiple tables via *joins* is inherent in any relatively normalized relational database of even modest size, and joins can be slow. The way that databases gain consistency is typically through the use of transactions, which require locking some portion of the database so it's not available to other clients. This can become untenable under very heavy loads, as the locks mean that competing users start queuing up, waiting for their turn to read or write the data.

You typically address these problems in one or more of the following ways, sometimes in this order:

- Throw hardware at the problem by adding more memory, adding faster processors, and upgrading disks. This is known as *vertical scaling*. This can relieve you for a time.
- When the problems arise again, the answer appears to be similar: now that one box is maxed out, you add hardware in the form of additional boxes in a database cluster. Now you have the problem of data replication and consistency during regular usage and in failover scenarios. You didn't have that problem before.
- Now you need to update the configuration of the database management system. This might mean optimizing the channels the database uses to write to the underlying filesystem. You turn off logging or journaling, which frequently is not a desirable (or, depending on your situation, legal) option.
- Having put what attention you could into the database system, you turn to your application. You try to improve your indexes. You optimize the queries. But presumably at this scale you weren't wholly ignorant of index and query optimization, and already had them in pretty good shape. So this becomes a painful process of picking through the data access code to find any opportunities for fine-tuning. This might include reducing or reorganizing joins, throwing out resource-intensive features such as XML processing within a stored procedure,

and so forth. Of course, presumably you were doing that XML processing for a reason, so if you have to do it somewhere, you move that problem to the application layer, hoping to solve it there and crossing your fingers that you don't break something else in the meantime.

- You employ a caching layer. For larger systems, this might include distributed caches such as Redis, memcached, Hazelcast, Aerospike, Ehcache, or Riak. Now you have a consistency problem between updates in the cache and updates in the database, which is exacerbated over a cluster.
- You turn your attention to the database again and decide that, now that the application is built and you understand the primary query paths, you can duplicate some of the data to make it look more like the queries that access it. This process, called denormalization, is antithetical to the five normal forms that characterize the relational model, and violates Codd's 12 Rules for relational data. You remind yourself that you live in this world, and not in some theoretical cloud, and then undertake to do what you must to make the application start responding at acceptable levels again, even if it's no longer "pure."



Codd's 12 Rules

Codd provided a list of 12 rules (there are actually 13, numbered 0 to 12) formalizing his definition of the relational model as a response to the divergence of commercial databases from his original concepts. Codd introduced his rules in a pair of articles in *CompuWorld* magazine in October 1985, and formalized them in the second edition of his book *The Relational Model for Database Management*, which is now out of print. Although Codd's rules represent an ideal system which commercial databases have typically implemented only partially, they have continued to exert a key influence over relational data modeling to the present day.

This likely sounds familiar to you. At web scale, engineers may legitimately ponder whether this situation isn't similar to Henry Ford's assertion that at a certain point, it's not simply a faster horse that you want. And they've done some impressive, interesting work.

We must therefore begin here in recognition that the relational model is simply a model. That is, it's intended to be a useful way of looking at the world, applicable to certain problems. It does not purport to be exhaustive, closing the case on all other ways of representing data, never again to be examined, leaving no room for alternatives. If you take the long view of history, Dr. Codd's model was a rather disruptive one in its time. It was new, with strange new vocabulary and terms such as *tuples*—familiar words used in a new and different manner. The relational model was held up

to suspicion, and doubtless suffered its vehement detractors. It encountered opposition even in the form of Dr. Codd's own employer, IBM, which had a very lucrative product set around IMS and didn't need a young upstart cutting into its pie.

But the relational model now arguably enjoys the best seat in the house within the data world. SQL is widely supported and well understood. It is taught in introductory university courses. Cloud-based Platform-as-a-Service (PaaS) providers such as Amazon Web Services, Google Cloud Platform, Microsoft Azure, Alibaba, and Rackspace provide relational database access as a service, including automated monitoring and maintenance features. Often the database you end up using is dictated by architectural standards within your organization. Even absent such standards, it's prudent to learn whatever your organization already has for a database platform. Your colleagues in development and infrastructure have considerable hard-won knowledge.

If by nothing more than osmosis (or inertia), you have learned over the years that a relational database is a one-size-fits-all solution.

So perhaps a better question is not, “**What's Wrong with Relational Databases?**” but rather, “What problem do you have?”

That is, you want to ensure that your solution matches the problem that you have. There are certain problems that relational databases solve very well. But the explosion of the web, and in particular social networks, means a corresponding explosion in the sheer volume of data you must deal with. When Tim Berners-Lee first worked on the web in the early 1990s, it was for the purpose of exchanging scientific documents between PhDs at a physics laboratory. Now, of course, the web has become so ubiquitous that it's used by everyone, from those same scientists to legions of five-year-olds exchanging emoji about kittens. That means in part that it must support enormous volumes of data; the fact that it does stands as a monument to the ingenious architecture of the web.

But as the traditional relational databases started to bend under the weight, it became clear that new solutions were needed.

A Quick Review of Relational Databases

Though you are likely familiar with them, let's briefly turn our attention to some of the foundational concepts in relational databases. This will give us a basis on which to consider more recent advances in thought around the trade-offs inherent in distributed data systems, especially very large distributed data systems, such as those that are required at web scale.

There are many reasons that the relational database has become so overwhelmingly popular over the last four decades. An important one is the Structured Query Language (SQL), which is feature-rich and uses a simple, declarative syntax. SQL was first

officially adopted as an American National Standards Institute (ANSI) standard in 1986; since that time, it's gone through several revisions and has also been extended with vendor-proprietary syntax such as Microsoft's T-SQL and Oracle's PL/SQL to provide additional implementation-specific features.

SQL is powerful for a variety of reasons. It allows the user to represent complex relationships with the data, using statements that form the Data Manipulation Language (DML) to insert, select, update, delete, truncate, and merge data. You can perform a rich variety of operations using functions based on relational algebra to find a maximum or minimum value in a set, for example, or to filter and order results. SQL statements support grouping aggregate values and executing summary functions. SQL provides a means of directly creating, altering, and dropping schema structures at runtime using Data Definition Language (DDL). SQL also allows you to grant and revoke rights for users and groups of users using the same syntax.

SQL is easy to use. The basic syntax can be learned quickly, and conceptually SQL and RDBMSs offer a low barrier to entry. Junior developers can become proficient readily, and as is often the case in an industry beset by rapid changes, tight deadlines, and exploding budgets, ease of use can be very important. And it's not just the syntax that's easy to use; there are many robust tools that include intuitive graphical interfaces for viewing and working with your database.

In part because it's a standard, SQL allows you to easily integrate your RDBMS with a wide variety of systems. All you need is a driver for your application language, and you're off to the races in a very portable way. If you decide to change your application implementation language (or your RDBMS vendor), you can often do that painlessly, assuming you haven't backed yourself into a corner using lots of proprietary extensions.

Transactions, ACID-ity, and Two-Phase Commit

In addition to the features mentioned already, RDBMSs and SQL also support *transactions*. A key feature of transactions is that they execute virtually at first, allowing the programmer to undo (using rollback) any changes that may have gone awry during execution; if all has gone well, the transaction can be reliably committed. As Jim Gray puts it, a transaction is “a transformation of state” that has the ACID properties (see “[The Transaction Concept: Virtues and Limitations](#)”).

ACID is an acronym for Atomic, Consistent, Isolated, Durable, which are the gauges you can use to assess that a transaction has executed properly and that it was successful:

Atomic

Atomic means “all or nothing”; that is, when a statement is executed, every update within the transaction must succeed in order to be called successful.

There is no partial failure where one update was successful and another related update failed. The common example here is with monetary transfers at an ATM: the transfer requires a debit from one account and a credit to another account. This operation cannot be subdivided; they must both succeed.

Consistent

Consistent means that data moves from one correct state to another correct state, with no possibility that readers could view different values that don't make sense together. For example, if a transaction attempts to delete a customer and their order history, it cannot leave order rows that reference the deleted customer's primary key; this is an inconsistent state that would cause errors if someone tried to read those order records.

Isolated

Isolated means that transactions executing concurrently will not become entangled with each other; they each execute in their own space. That is, if two different transactions attempt to modify the same data at the same time, then one of them will have to wait for the other to complete.

Durable

Once a transaction has succeeded, the changes will not be lost. This doesn't imply another transaction won't later modify the same data; it just means that writers can be confident that the changes are available for the next transaction to work with as necessary.

The debate about support for transactions comes up very quickly as a sore spot in conversations around nonrelational data stores, so let's take a moment to revisit what this really means. On the surface, ACID properties seem so obviously desirable as to not even merit conversation. Presumably no one who runs a database would suggest that data updates don't have to endure for some length of time; that's the very point of making updates—that they're there for others to read. However, a more subtle examination might lead you to want to find a way to tune these properties a bit and control them slightly. There is, as they say, no free lunch on the internet, and once you see how you're paying for transactions, you may start to wonder whether there's an alternative.

Transactions become difficult under heavy load. When you first attempt to horizontally scale a relational database, making it distributed, you must now account for *distributed transactions*, where the transaction isn't simply operating inside a single table or a single database, but is spread across multiple systems. In order to continue to honor the ACID properties of transactions, you now need a transaction manager to orchestrate across the multiple nodes.

In order to account for successful completion across multiple hosts, the idea of a two-phase commit (sometimes referred to as "2PC") is introduced. The two-phase

commit is a commonly used algorithm for achieving consensus in distributed systems, involving two sets of interactions between hosts known as the prepare phase and commit phase. Because the two-phase commit locks all associated resources, it is useful only for operations that can complete very quickly. Although it may often be the case that your distributed operations can complete in subsecond time, it is certainly not always the case. Some use cases require coordination between multiple hosts that you may not control yourself. Operations coordinating several different but related activities can take hours to update.

Two-phase commit *blocks*; that is, clients (“competing consumers”) must wait for a prior transaction to finish before they can access the blocked resource. The protocol will wait for a node to respond, even if it has died. It’s possible to avoid waiting forever in this event, because a timeout can be set that allows the transaction coordinator node to decide that the node isn’t going to respond and that it should abort the transaction. However, an infinite loop is still possible with 2PC; that’s because a node can send a message to the transaction coordinator node agreeing that it’s OK for the coordinator to commit the entire transaction. The node will then wait for the coordinator to send a commit response (or a rollback response if, say, a different node can’t commit); if the coordinator is down in this scenario, that node conceivably will wait forever.

So in order to account for these shortcomings in two-phase commit of distributed transactions, the database world turned to the idea of *compensation*. Compensation, often used in web services, means in simple terms that the operation is immediately committed, and then in the event that some error is reported, a new operation is invoked to restore proper state.

There are a few basic, well-known patterns for compensatory action that architects frequently have to consider as an alternative to two-phase commit. These include writing off the transaction if it fails, deciding to discard erroneous transactions and reconciling later. Another alternative is to retry failed operations later on notification. In a reservation system or a stock sales ticker, these are not likely to meet your requirements. For other kinds of applications, such as billing or ticketing applications, this can be acceptable.



The Problem with Two-Phase Commit

Gregor Hohpe, a Google architect, wrote a wonderful and often-cited blog entry titled “[Starbucks Does Not Use Two-Phase Commit](#)”. It shows in real-world terms how difficult it is to scale two-phase commit and highlights some of the alternatives that are mentioned here. It’s an easy, fun, and enlightening read. If you’re interested in digging deeper, Martin Kleppman’s comprehensive book *[Designing Data-Intensive Applications](#)* (O’Reilly) contains an excellent in-depth discussion of two-phase commit and other consensus algorithms.

The problems that 2PC introduces for application developers include loss of availability and higher latency during partial failures. Neither of these is desirable. So once you’ve had the good fortune of being successful enough to necessitate scaling your database past a single machine, you now have to figure out how to handle transactions across multiple machines and still make the ACID properties apply. Whether you have 10 or 100 or 1,000 database machines, atomicity is still required in transactions as if you were working on a single node. But it’s now a much, much bigger pill to swallow.

Schema

One often-lauded feature of relational database systems is the rich schemas they afford. You can represent your domain objects in a relational model. A whole industry has sprung up around (expensive) tools such as the CA ERwin Data Modeler to support this effort. In order to create a properly normalized schema, however, you are forced to create tables that don’t exist as business objects in your domain. For example, a schema for a university database might require a “student” table and a “course” table. But because of the “many-to-many” relationship here (one student can take many courses at the same time, and one course has many students at the same time), you have to create a join table. This pollutes a pristine data model, where you’d prefer to just have students and courses. It also forces you to create more complex SQL statements to join these tables together. The join statements, in turn, can be slow.

Again, in a system of modest size, this isn’t much of a problem. But complex queries and multiple joins can become burdensomely slow once you have a large number of rows in many tables to handle.

Finally, not all schemas map well to the relational model. One type of system that has risen in popularity in the last decade is the *complex event processing system* or *stream processing system*, which represents state changes in a very fast stream. It’s often useful to contextualize events at runtime against other events that might be related in order to infer some conclusion to support business decision-making. Although event

streams can be represented in terms of a relational database, as with Apache Kafka's KSQL, it is often an uncomfortable stretch.

If you're an application developer, you'll no doubt be familiar with the many object-relational mapping (ORM) frameworks that have sprung up in recent years to help ease the difficulty in mapping application objects to a relational model. Again, for small systems, ORM can be a relief. But it also introduces new problems of its own, such as extended memory requirements, and it often pollutes the application code with increasingly unwieldy mapping code. Here's an example of a Java method using Hibernate to "ease the burden" of having to write the SQL code:

```
@CollectionOfElements
@JoinTable(name="store_description",
    joinColumns = @JoinColumn(name="store_code"))
@MapKey(columns={@Column(name="for_store",length=3)})
@Column(name="description")
private Map<String, String> getMap() {
    return this.map;
}
//... etc.
```

Is it certain that we've done anything but move the problem here? Of course, with some systems, such as those that make extensive use of document exchange, as with services or XML-based applications, there are not always clear mappings to a relational database. This exacerbates the problem.

Sharding and Shared-Nothing Architecture

If you can't split it, you can't scale it.

—Randy Shoup, Distinguished Architect, eBay

Another way to attempt to scale a relational database is to introduce *sharding* to your architecture. This has been used to good effect at large websites such as eBay, which supports billions of SQL queries a day, and in other modern web applications. The idea here is that you split the data so that instead of hosting all of it on a single server or replicating all of the data on all of the servers in a cluster, you divide up portions of the data horizontally and host them each separately.

For example, consider a large customer table in a relational database. The least disruptive thing (for the programming staff, anyway) is to vertically scale by adding CPU, adding memory, and getting faster hard drives, but if you continue to be successful and add more customers, at some point (perhaps into the tens of millions of rows), you'll likely have to start thinking about how you can add more machines. When you do so, do you just copy the data so that all of the machines have it? Or do you instead divide up that single customer table so that each database has only some of the records, with their order preserved? Then, when clients execute queries, they

put load only on the machine that has the record they're looking for, with no load on the other machines.

It seems clear that in order to shard, you need to find a good key by which to order your records. For example, you could divide your customer records across 26 machines, one for each letter of the alphabet, with each hosting only the records for customers whose last names start with that particular letter. It's likely this is not a good strategy, however—there probably aren't many last names that begin with “Q” or “Z,” so those machines will sit idle while the “J,” “M,” and “S” machines spike. You could shard according to something numeric, like phone number, “member since” date, or the name of the customer's state. It all depends on how your specific data is likely to be distributed.

There are three basic strategies for determining shard structure:

Feature-based shard or functional segmentation

This is the approach taken by Randy Shoup, Distinguished Architect at eBay, who in 2006 helped bring the site's architecture into maturity to support many billions of queries per day. Using this strategy, the data is split not by dividing records in a single table (as in the customer example discussed earlier), but rather by splitting into separate databases the features that don't overlap with each other very much. For example, at eBay, the users are in one shard, and the items for sale are in another. This approach depends on understanding your domain so that you can segment data cleanly.

Key-based sharding

In this approach, you find a key in your data that will evenly distribute it across shards. So instead of simply storing one letter of the alphabet for each server as in the (naive and improper) earlier example, you use a one-way hash on a key data element and distribute data across machines according to the hash. It is common in this strategy to find time-based or numeric keys to hash on.

Lookup table

In this approach, also known as *directory-based sharding*, one of the nodes in the cluster acts as a “Yellow Pages” directory and looks up which node has the data you're trying to access. This has two obvious disadvantages. The first is that you'll take a performance hit every time you have to go through the lookup table as an additional hop. The second is that the lookup table not only becomes a bottleneck, but a single point of failure.

Sharding can minimize contention depending on your strategy and allows you not just to scale horizontally, but then to scale more precisely, as you can add power to the particular shards that need it.

Sharding could be termed a kind of *shared-nothing* architecture that's specific to databases. A shared-nothing architecture is one in which there is no centralized (shared)

state, but each node in a distributed system is independent, so there is no client contention for shared resources.

Shared-nothing architecture was more recently popularized by Google, which has written systems such as its Bigtable database and its MapReduce implementation that do not share state, and are therefore capable of near-infinite scaling. The Cassandra database is a shared-nothing architecture, as it has no central controller and no notion of primary/secondary replicas; all of its nodes are the same.



More on Shared-Nothing Architecture

The term was first coined by Michael Stonebraker at the University of California at Berkeley in his 1986 paper “[The Case for Shared Nothing](#).” It’s only a few pages. If you take a look, you’ll see that many of the features of shared-nothing distributed data architecture, such as ease of high availability and the ability to scale to a very large number of machines, are the very things that Cassandra excels at.

Many nonrelational databases offer this automatically and out of the box is very handy; creating and maintaining custom data shards by hand is a wicked proposition. For example, MongoDB, which we’ll discuss later, provides auto-sharding capabilities to manage failover and node balancing. It’s good to understand sharding in terms of data architecture in general, but especially in terms of Cassandra more specifically. Cassandra uses an approach similar to key-based sharding to distribute data across nodes, but does so automatically.

Web Scale

In summary, relational databases are very good at solving certain data storage problems, but because of their focus, they also can create problems of their own when it’s time to scale. Then, you often need to find a way to get rid of your joins, which means denormalizing the data, which means maintaining multiple copies of data and seriously disrupting your design, both in the database and in your application. Further, you almost certainly need to find a way around distributed transactions, which will quickly become a bottleneck. These compensatory actions are not directly supported in any but the most expensive RDBMSs. And even if you can write such a huge check, you still need to carefully choose partitioning keys to the point where you can never entirely ignore the limitation.

Perhaps more importantly, as you see some of the limitations of RDBMSs and consequently some of the strategies that architects have used to mitigate their scaling issues, a picture slowly starts to emerge. It’s a picture that makes some NoSQL solutions seem perhaps less radical and less scary than you may have thought at first,

and more like a natural expression and encapsulation of some of the work that was already being done to manage very large databases.

Because of some of the inherent design decisions in RDBMSs, it is not always as easy to scale as some other, more recent possibilities that take the structure of the web into consideration. However, it's not only the structure of the web you need to consider, but also its phenomenal growth, because as more and more data becomes available, you need architectures that allow your organization to take advantage of this data in near real time to support decision making, and to offer new and more powerful features and capabilities to your customers.



Data Scale, Then and Now

It has been said, though it is hard to verify, that the 17th-century English poet John Milton had actually read every published book on the face of the earth. Milton knew many languages (he was even learning Navajo at the time of his death), and given that the total number of published books at that time was in the thousands, this would have been possible. The size of the world's data stores have grown somewhat since then.

With the rapid growth in the web, there is great variety to the kinds of data that need to be stored, processed, and queried, and some variety to the businesses that use such data. Consider not only customer data at familiar retailers or suppliers, and not only digital video content, but also the required move to digital television and the explosive growth of email, messaging, mobile phones, RFID, Voice Over IP (VoIP) usage, and the Internet of Things (IoT). Companies that provide content—and the third-party value-add businesses built around them—require very scalable data solutions. Consider too that a typical business application developer or database administrator may be used to thinking of relational databases as the center of the universe. You might then be surprised to learn that within corporations, around 80% of data is unstructured.

The Rise of NoSQL

The recent interest in nonrelational databases reflects the growing sense of need in the software development community for web scale data solutions. The term “NoSQL” began gaining popularity around 2009 as a shorthand way of describing these databases. The term has historically been the subject of much debate, but a consensus has emerged that the term refers to nonrelational databases that support “not only SQL” semantics.

Various experts have attempted to organize these databases in a few broad categories; let's examine a few of the most common:

Key-value stores

In a key-value store, the data items are keys that have a set of attributes. All data relevant to a key is stored with the key; data is frequently duplicated. Popular key-value stores include Amazon's Dynamo DB, Riak, and Voldemort. Additionally, many popular caching technologies act as key-value stores, including Oracle Coherence, Redis, and Memcached.

Column stores

In a column store, also known as a *wide-column store* or *column-oriented store*, data is stored by column rather than by row. For example, in a column store, all customer addresses might be stored together, allowing them to be retrieved in a single query. Popular column stores include Apache Hadoop's HBase, Apache Kudu, and Apache Druid.

Document stores

The basic unit of storage in a document database is the complete document, often stored in a format such as JSON, XML, or YAML. Popular document stores include MongoDB, CouchDB, and several public cloud offerings.

Graph databases

Graph databases represent data as a graph—a network of nodes and edges that connect the nodes. Both nodes and edges can have properties. Because they give heightened importance to relationships, graph databases such as Neo4j, Janus-Graph, and DataStax Graph have proven popular for building social networking and semantic web applications.

Object databases

Object databases store data not in terms of relations and columns and rows, but in terms of objects as understood from the discipline of object-oriented programming. This makes it straightforward to use these databases from object-oriented applications. Object databases such as db4o and InterSystems Caché allow you to avoid techniques like stored procedures and object-relational mapping (ORM) tools. The most widely used object database is Amazon Web Services' Simple Storage Service (S3).

XML databases

XML databases are a special form of document databases, optimized specifically for working with data described in the eXtensible Markup Language (XML). So-called "XML native" databases include BaseX and eXist.

Multimodel databases

Databases that support more than one of these styles have been growing in popularity. These "multimodel" databases are based on a primary underlying database (most often a relational, key-value, or column store) and expose additional models as APIs on top of that underlying database. Examples of these include

Microsoft Azure Cosmos DB, which exposes document, wide column, and graph APIs on top of a key-value store, and DataStax Enterprise, which offers a graph API on top of Cassandra's wide column model. Multimodel databases are often touted for their ability to support an approach known as *polyglot persistence*, in which different microservices or components of an application can interact with data using more than one of the models we've described here. We'll discuss an example of polyglot persistence in [Chapter 7](#).



Learning More About NoSQL Databases

For a comprehensive list of NoSQL databases, see the [NoSQL site](#). The [DB-Engines site](#) also provides popularity rankings of popular databases by type, updated monthly.

There is wide variety in the goals and features of these databases, but they tend to share a set of common characteristics. The most obvious of these is implied by the name NoSQL—these databases support data models, Data Definition Languages (DDLs), and interfaces beyond the standard SQL available in popular relational databases. In addition, these databases are typically distributed systems without centralized control. They emphasize horizontal scalability and high availability, in some cases at the cost of strong consistency and ACID semantics. They tend to support rapid development and deployment. They take flexible approaches to schema definition, in some cases not requiring any schema to be defined up front. They provide support for Big Data and analytics use cases.

Over the past decade, there have been a large number of open source and commercial offerings in the NoSQL space. The adoption and quality of these have varied widely, but leaders have emerged in the categories just discussed, and many have become mature technologies with large installation bases and commercial support. We're happy to report that Cassandra is one of those technologies, as we'll dig into more in the next chapter.

New Relational Architectures and NewSQL

Many of the challenges with previous approaches to scale relational databases that we've described in this chapter can be attributed to designs that attempted to graft distributed systems principles on top of existing database engines, with generally unsatisfactory results.

In response to the criticisms of traditional RDBMS that we've summarized in this chapter, database researchers began to explore new approaches for creating more scalable relational systems. In 2012, two key papers were published, proposing new approaches for providing transactional guarantees at scale. The first was the [Calvin transaction protocol](#) developed at Yale University described an approach based on a

global consensus protocol used by all transactions on a database. FaunaDB is an example of a database that implements the approach on the Calvin paper.

Google's [Spanner paper](#), published a couple of months later, proposed an approach in which the database is divided into shards and a separate consensus protocol is applied to shards to support transactional guarantees. Google Cloud Spanner, CockroachDB, and YugaByteDB are examples of databases that follow this approach.

Don't worry if the references to consistency don't make sense yet; we'll dive into this more in "[Brewer's CAP Theorem](#)" on [page 23](#). The main takeaway for now is that these so called *NewSQL* databases were designed from the ground up to support ACID transaction semantics at scale. Note that the reference to SQL is possibly misleading, as not all of these databases provide full ANSI SQL support.

Summary

The relational model has served the software industry well over the past four decades, but the level of availability and scalability required for modern applications has stretched traditional relational database technology to the breaking point.

The intention of this book is not to convince you by clever argument to adopt a non-relational database such as Apache Cassandra. It is only our intention to present what Cassandra can do and how it does it so that you can make an informed decision and get started working with it in practical ways if you find it applies.

Perhaps the ultimate question, then, is not "[What's Wrong with Relational Databases?](#)" but rather, "What kinds of things would I do with data if it wasn't a problem?" In a world now working at web scale and looking to the future, Apache Cassandra might be one part of the answer.

Introducing Cassandra

An invention has to make sense in the world in which it is finished, not the world in which it is started.

—Ray Kurzweil

In the previous chapter, we discussed the emergence of nonrelational database technologies in order to meet the increasing demands of modern web scale applications. In this chapter, we'll focus on Cassandra's value proposition and key tenets to show how it rises to the challenge. You'll also learn about Cassandra's history and how you can get involved in the open source community that maintains Cassandra.

The Cassandra Elevator Pitch

Hollywood screenwriters and software entrepreneurs are often advised to have their “elevator pitch” ready. This is a summary of exactly what their product is all about—concise, clear, and brief enough to deliver in just a minute or two, in the lucky event that they find themselves sharing an elevator with an executive, agent, or investor who might consider funding their project. Cassandra has a compelling story, so let's boil it down to an elevator pitch that you can present to your manager or colleagues should the occasion arise.

Cassandra in 50 Words or Less

“Apache Cassandra is an open source, distributed, decentralized, elastically scalable, highly available, fault-tolerant, tuneably consistent, row-oriented database. Cassandra bases its distribution design on Amazon's Dynamo and its data model on Google's Bigtable, with a query language similar to SQL. Created at Facebook, it now powers cloud-scale applications across many industries.” That's exactly 50 words.

Of course, if you were to recite that to your boss in the elevator, you'd probably get a blank look in return. So let's break down the key points in the following sections.

Distributed and Decentralized

Cassandra is *distributed*, which means that it is capable of running on multiple machines while appearing to users as a unified whole. In fact, there is little point in running a single Cassandra node. Although you can do it, and that's acceptable for getting up to speed on how it works, you quickly realize that you'll need multiple machines to really realize any benefit from running Cassandra. Much of its design and codebase is specifically engineered toward not only making it work across many different machines, but also for optimizing performance across multiple data center racks, and even for a single Cassandra cluster running across geographically dispersed data centers. You can confidently write data to anywhere in the cluster and Cassandra will get it.

Once you start to scale many other data stores (MySQL, Bigtable), some nodes need to be set up as primary replicas in order to organize other nodes, which are set up as secondary replicas. Cassandra, however, is decentralized, meaning that every node is identical; no Cassandra node performs certain organizing operations distinct from any other node. Instead, Cassandra features a peer-to-peer architecture and uses a gossip protocol to maintain and keep in sync a list of nodes that are alive or dead. We'll discuss this more in [“Gossip and Failure Detection” on page 108](#).

The fact that Cassandra is *decentralized* means that there is no single point of failure. All of the nodes in a Cassandra cluster function exactly the same. This is sometimes referred to as “server symmetry.” Because they are all doing the same thing, by definition there can't be a special host that is coordinating activities, as with the primary/secondary setup that you see in MySQL, Bigtable, and so many other databases.

In many distributed data solutions (such as RDBMS clusters), you set up multiple copies of data on different servers in a process called replication, which copies the data to multiple machines so that they can all serve simultaneous requests and improve performance. Typically this process is not decentralized, as in Cassandra, but is rather performed by defining a *primary/secondary relationship*. That is, all of the servers in this kind of cluster don't function in the same way. You configure your cluster by designating one server as the primary (or *primary replica*) and others as *secondary replicas*. The primary replica acts as the authoritative source of the data, and operates in a unidirectional relationship with the secondary replicas, which must synchronize their copies. If the primary node fails, the whole database is in jeopardy. To work around the situation of the primary as a single point of failure, you often need to add complexity to the environment in the form of multiple primary nodes. Note that while we frequently understand primary/secondary replication in the RDBMS world, there are NoSQL databases such as MongoDB that follow the

primary/secondary scheme as well. Even Mongo’s “replica set” mechanism is essentially a primary/secondary scheme in which the primary can be replaced by an automated leader election process.

Decentralization, therefore, has two key advantages: it’s simpler to use than primary/secondary, and it helps you avoid outages. It is simpler to operate and maintain a decentralized store than a primary/secondary store because all nodes are the same. That means that you don’t need any special knowledge to scale; setting up 50 nodes isn’t much different from setting up one. There’s next to no configuration required to support it. Because all of the replicas in Cassandra are identical, failures of a node won’t disrupt service.

In short, because Cassandra is distributed and decentralized, there is no single point of failure, which supports high availability.

Elastic Scalability

Scalability is an architectural feature of a system that can continue serving a greater number of requests with little degradation in performance. *Vertical scaling*—simply adding more processing capacity and memory to your existing machine—is the easiest way to achieve this. *Horizontal scaling* means adding more machines that have all or some of the data on them so that no one machine has to bear the entire burden of serving requests. But then the software itself must have an internal mechanism for keeping its data in sync with the other nodes in the cluster.

Elastic scalability refers to a special property of horizontal scalability. It means that your cluster can seamlessly scale up and scale back down. To do this, the cluster must be able to accept new nodes that can begin participating by getting a copy of some or all of the data and start serving new user requests without major disruption or reconfiguration of the entire cluster. You don’t have to restart your process. You don’t have to change your application queries. You don’t have to manually rebalance the data yourself. Just add another machine—Cassandra will find it and start sending it work.

Scaling down, of course, means removing some of the processing capacity from your cluster. You might do this for business reasons, such as adjusting to seasonal workloads in retail or travel applications. Or perhaps there will be technical reasons such as moving parts of your application to another platform. As much as we try to minimize these situations, they still happen. But when they do, you won’t need to upset the entire apple cart to scale back.

High Availability and Fault Tolerance

In general architecture terms, the availability of a system is measured according to its ability to fulfill requests. But computers can experience all manner of failure, from hardware component failure to network disruption to corruption. Any computer is

susceptible to these kinds of failure. There are of course very sophisticated (and often prohibitively expensive) computers that can themselves mitigate many of these circumstances, as they include internal hardware redundancies and facilities to send notification of failure events and hot swap components. But anyone can accidentally break an Ethernet cable, and catastrophic events can beset a single data center. So for a system to be highly available, it must typically include multiple networked computers, and the software they're running must then be capable of operating in a cluster and have some facility for recognizing node failures and failing over requests to another part of the system.

Cassandra is highly available. You can replace failed nodes in the cluster with no downtime, and you can replicate data to multiple data centers to offer improved local performance and prevent downtime if one data center experiences a catastrophe such as fire or flood.

Tuneable Consistency

Consistency is an overloaded term in the database world, but for our purposes we will use the definition that a read always returns the most recently written value. Consider the case of two customers attempting to put the same item into their shopping carts on an ecommerce site. If I place the last item in stock into my cart an instant after you do, you should get the item added to your cart, and I should be informed that the item is no longer available for purchase. This is guaranteed to happen when the state of a write is consistent among all nodes that have that data.

But as we'll see later, scaling data stores means making certain trade-offs among data consistency, node availability, and partition tolerance. Cassandra is frequently called "eventually consistent," which is a bit misleading. Out of the box, Cassandra trades some consistency in order to achieve total availability. But Cassandra is more accurately termed "tuneably consistent," which means it allows you to easily decide the level of consistency you require, in balance with the level of availability.

Let's take a moment to unpack this, as the term "eventual consistency" has caused some uproar in the industry. Some practitioners hesitate to use a system that is described as "eventually consistent."

For detractors of eventual consistency, the broad argument goes something like this: eventual consistency is maybe OK for social web applications where data doesn't *really* matter. After all, you're just posting to Mom what little Billy ate for breakfast, and if it gets lost, it doesn't really matter. But the data *I* have is actually really important, and it's ridiculous to think that I could allow eventual consistency in my model.

Set aside the number of large-scale web applications (Amazon, Facebook, Google, Twitter) that use this model, and perhaps there's something to this argument. Presumably such data is very important indeed to the companies running these